In Class Lab 5f

Your First Stored Procedures | CIS 4720 Database Management and SQL

Points	Time	Work in	Turn in
10	About 90 minutes	lab5f_starter.sql	Your GitHub repository URL

Textbook: Coronel and Morris, *Database Systems: Design, Implementation, and Management*, 14th edition, Chapter 5, Section 5.8.
The due date is listed on Canvas LMS.

What this lab adds to Lab 5e

You have now written two kinds of procedural SQL. In Lab 5d you wrote a **function**, calculate_gpa, which works out one value and hands it back. In Lab 5e you wrote **triggers**, which run by themselves whenever a row changes. This lab adds the third kind: the **stored procedure**.

A stored procedure is a named block of SQL statements saved inside the database. It runs only when somebody asks for it by name, with the keyword **CALL**. Think of it as a saved job: instead of typing the same SELECT every time, you save it once and call it.

	Function	Trigger	Stored procedure
How it runs	Inside another statement, as in SELECT calculate_gpa(1)	By itself, when a row is inserted, updated or deleted	When you CALL it by name
What it gives back	Exactly one value	Nothing	Rows, like a SELECT, or nothing
Yours so far	calculate_gpa	trg_attendance_before_insert, trg_scores_after_delete	None yet. Four by the end of today

Why databases have them

A real application, such as the registrar's website, does not send long SELECT statements to the database. It sends something short like **CALL student_report(3)**. The SQL lives in one place, inside the database, so it is written once, tested once, and fixed once when something changes. Every program that needs a student report calls the same procedure.

Part	What you do	Steps
1	Set up from the starter file	1 to 2
2	Write and call procedures, with and without an IN parameter	3 to 5
3	Change a procedure, and write one completely on your own	6 to 7
4	Tell a procedure from a function, and list what you built	8 to 9
5	Push to GitHub	10 to 11

Part 1. Get set up

Same routine as Lab 5e. SQLTools will not connect to a schema until one exists, so you connect to one that is always there and use it to make your own.

Step 1. Connect to information_schema and make your folder

Make a folder on OneDrive called **CIS4720_Lab5f** through the OneDrive app, download **lab5f_starter.sql** from Canvas into it, and open that folder in VS Code with **File**, then **Open Folder**.

Open the SQLTools panel, choose **Add New Connection** and pick **MySQL**. Server **localhost**, port **3306**, username **root**, your password, and for the database use **information_schema**. Test it, then save it.

Step 2. Create the schema and run the starter file

Open lab5f_starter.sql. Run the first three statements one at a time:

```
show databases;

create database if not exists cis_students_db;

use cis_students_db;
```

Then add a second SQLTools connection with **Database** set to **cis_students_db** and work from that one for the rest of the lab. Run the rest of the starter file one statement at a time. The function and the two triggers are each one statement from CREATE down to END, so select the whole block and run the selection.

Then check you landed where Lab 5e left off:

Check	What you should see
SELECT * FROM students	Five rows, gpa 85.00, 84.00, 92.33, 82.50 and 0.00
SELECT * FROM attendance	Four rows, ids 500 to 503. The last one has today's date and PRESENT in capitals

> **Check where you are before you build anything**
>
> Run **SELECT DATABASE();** at any point. It names the schema your statements are landing in. If it says information_schema or NULL, fix that before writing anything else.

Part 2. Writing and calling procedures

Step 3. Your first procedure, list_students

This one is printed in full, because every part of it is new. It has no parameters and holds one SELECT. Copy it under the Step 3 heading, select the whole block and run the selection.

```
-- a saved job that lists every student
CREATE PROCEDURE list_students()
BEGIN
    -- the statement the procedure runs each time it is called
    SELECT student_id, first_name, last_name, gpa
    FROM students;
END
```

Piece	What it means
CREATE PROCEDURE list_students	Save a procedure in this schema under this name
()	The parameter list. Empty, because this procedure needs nothing from you. The brackets are still required
BEGIN ... END	The body. Everything between them runs, in order, each time the procedure is called
The ; after FROM students	Ends the SELECT inside the body. It does not end the procedure. END does that

Creating the procedure does not run it. Nothing comes back yet. To run it, call it:

```
-- run the saved job
CALL list_students();
```

Five rows come back, the same five a SELECT * FROM students would give you, without the SELECT. Call it twice more. It does the same job every time.

> **No DELIMITER anywhere in this lab**
>
> A procedure is one statement, however many semicolons are inside it. Select the whole block from CREATE PROCEDURE down to END and run the selection.
>
> The textbook and most web pages wrap procedures in **DELIMITER $$** and **DELIMITER ;** with **$$** after END. That is an instruction to the command prompt program, not SQL. In VS Code, delete all of it or the server rejects it with **error 1064**.
>
> A comment needs a space after the two dashes. **-- note** is a comment, **--note** is a syntax error.

Step 4. A procedure with an IN parameter, student_report

list_students always does exactly the same thing. Most useful procedures need to be told something first, such as which student. That is what a **parameter** is for.

```
-- shows every test score for the one student you ask about
CREATE PROCEDURE student_report(IN p_student_id INT)
BEGIN
    -- choose the three columns from the view, for one student only
    -- >>>>>  YOUR STATEMENT GOES HERE  <<<<<
END
```

Piece	What it means
IN	The value travels into the procedure from the CALL. You will meet the other direction in a later lab

Piece	What it means
p_student_id	The parameter's name. The p_ at the front is a habit that stops it being confused with the student_id column
INT	Its data type, the same as the column it will be compared with

Writing the statement that goes in the gap

One SELECT, and you have written dozens of them. The only new idea is where the value in the WHERE clause comes from.

1. SELECT three columns: **student_name**, **test_name** and **score**.

2. FROM the **student_scores** view you built in Lab 5d. A procedure can read a view exactly as it reads a table.

3. WHERE the view's student_id column equals **p_student_id**. No quotes around it. It is a name, and whatever number was passed in the CALL takes its place.

4. End the SELECT with a semicolon, then leave END exactly as it is.

Leave the WHERE off and the procedure ignores what you asked for and reports on everybody.

Step 5. Call student_report three times

You write these three CALL statements. The number you are asking about goes inside the brackets. Call it for student **3**, then **5**, then **9**.

CALL for	What comes back	Why
Student 3	Three rows for Maria Elena Jimenez: quiz1 95, quiz2 88, quiz3 94	She has three scores
Student 5	One row: Rita Nunez, with NULL test_name and NULL score	Rita has no scores yet. The view uses a LEFT OUTER JOIN, so she still appears
Student 9	No rows at all, and no error	There is no student 9. The procedure did exactly what it was told

Keep that last row in mind. Asking about a student who does not exist quietly gives you nothing back, which is easy to mistake for a student with no scores. In a later lab you will teach a procedure to notice that and say so.

Part 3. Changing a procedure, and writing one yourself

Step 6. Change list_students to put the highest gpa first

First try the obvious thing: run your Step 3 CREATE PROCEDURE block again. The server refuses with **ERROR 1304, PROCEDURE list_students already exists**. Put that error in a comment.

MySQL has no statement for editing the body of a procedure. The way to change one is to drop it and create it again. You write both statements for this step:

1. Drop the procedure called **list_students**, adding **IF EXISTS** so the statement is safe to run twice. It is built exactly like the DROP TRIGGER statement from Lab 5e, with one word changed.

2. Create list_students again, the same as Step 3 but with the rows sorted by **gpa**, highest first. Remember where ORDER BY goes and which word makes it count down.

Call it. The order should now be:

student_id	first_name	last_name	gpa
3	Maria Elena	Jimenez	92.33
1	Jenna	Wilson	85.00
2	Robert	Reese	84.00
4	Inder	Singh	82.50
5	Rita	Nunez	0.00

Notice what did not change: the name, and the CALL. Anybody who calls list_students now gets the new order without changing anything on their side. That is the point of keeping the SQL inside the database.

Step 7. Write scores_for_test completely on your own

Everything for this one is in Steps 3 to 6. Create a procedure named **scores_for_test** that takes one IN parameter, the name of a test, and lists everybody's score on that test.

Requirement	Detail
Parameter	One IN parameter called p_test_name, type VARCHAR(20), the same type as the test_name column
Columns	student_name and score, from the student_scores view
Rows	Only the test named in the parameter
Order	Highest score first

Then call it three times, for **quiz2**, **quiz3** and **quiz9**. A test name is text, so when you call it the value goes inside single quotes, the same as text anywhere else in SQL. Inside the procedure, p_test_name has no quotes.

CALL for	What comes back
quiz2	Four rows: Inder Singh 91, Maria Elena Jimenez 88, Jenna Wilson 80, Robert Reese 72
quiz3	Three rows: Robert Reese 95, Maria Elena Jimenez 94, Inder Singh 74
quiz9	No rows. There is no quiz9

Part 4. Procedure or function?

Step 8. Three deliberate mistakes

Functions and procedures look alike on the page, so the most common early mistake is calling one the way you call the other. Run these three on purpose. They are printed for you, because the point is the error, not the typing. Copy each error number and message into a comment under the statement.

```
-- a procedure used like a function
SELECT student_report(1);

-- a function used like a procedure
CALL calculate_gpa(1);

-- a procedure called without the value it needs
CALL student_report();
```

Statement	Error	What it is telling you
SELECT student_report(1)	1305, FUNCTION ... student_report does not exist	SELECT looks for a function. There is no function by that name, only a procedure
CALL calculate_gpa(1)	1305, PROCEDURE ... calculate_gpa does not exist	CALL looks for a procedure. calculate_gpa is a function
CALL student_report()	1318, incorrect number of arguments, expected 1, got 0	Every IN parameter must be given a value

The rule to remember

A **function** is used inside a statement, usually a SELECT or an UPDATE, and hands back one value. A **procedure** stands on its own and is started with **CALL**. If the server says something does not exist and you are sure you created it, check which kind it is.

Step 9. See what you have built

Functions and procedures together are called **routines**, and the server keeps a list of them in its catalogue. This query is given to you:

```
-- every function and procedure in this schema
SELECT routine_name, routine_type
FROM information_schema.routines
WHERE routine_schema = 'cis_students_db'
ORDER BY routine_name;
```

routine_name	routine_type
calculate_gpa	FUNCTION
list_students	PROCEDURE
scores_for_test	PROCEDURE
student_report	PROCEDURE

Then write one statement yourself: show the full definition of **scores_for_test**. In Lab 5d you used SHOW CREATE FUNCTION. The procedure version is built the same way. Widen the Create Procedure cell in the results, or click into it, to read the whole body.

Part 5. Push to GitHub

Step 10. Publish to a new repository

Save lab5f_starter.sql first. If you are not signed in to GitHub, click **Accounts** at the bottom of the left bar, choose **Sign in with GitHub**, and click **Authorize**.

1. Click **Source Control**, then **Publish to GitHub**.
2. Name it **cis4720-lab5f**. It has to be a name you have not used.
3. Choose **Publish to a public repository**. A private one gives your instructor a 404 page instead of your work.
4. Tick **lab5f_starter.sql**. Leave the **.vscode** folder unticked, because that is the one holding your password.

> **Sign out of GitHub before you leave the lab**
>
> Shared PC. Your sign in stays on it until somebody removes it. Click **Accounts**, then your username, then **Sign Out**. Everything else on the PC is wiped tonight, so the repository is the only copy of your work.

Step 11. Check it and submit the URL

1. Open the repository on github.com, click the file and read it on the page. What is there is exactly what gets graded.
2. Open the URL in a private or incognito window. A **404** means it is private, so go to **Settings** and change the visibility to public.
3. Paste the URL into the Lab 5f submission box on Canvas.

Submit the URL even if you did not finish. Partial work your instructor can read scores better than finished work nobody can reach.

How it is graded

Steps	What is being checked	Points
1 to 2	Schema created and the starter file running	1.0
3	list_students created and called	1.0
4	student_report created, with the SELECT you wrote	1.5
5	student_report called for students 3, 5 and 9	1.0
6	list_students dropped and recreated, highest gpa first, with the 1304 error noted	1.5
7	scores_for_test written on your own and called three times	2.0
8	The three errors run and recorded in comments	0.5
9	The routines listed and SHOW CREATE PROCEDURE run	0.5
All	Required. A comment in your own words above every statement	1.0
10 to 11	A new public repository with the file in it, URL submitted	0.0

Total 10 points. The due date is listed on Canvas LMS.

If something goes wrong

What you see	What it means
ERROR 1064 near 'DELIMITER $$'	You copied a procedure that still has the DELIMITER lines around it. Delete them and the $$ after END.
ERROR 1064 near '--something'	A comment lost the space after its two dashes. Put the space back, or use # instead.
ERROR 1064 near the end of the SELECT	VS Code ran only part of the block. Select the whole block from CREATE PROCEDURE to END and run the selection.
ERROR 1304, PROCEDURE already exists	You ran a CREATE twice. Drop the procedure, then create it again.
ERROR 1305, does not exist	Either it is a function and you used CALL, it is a procedure and you used SELECT, or it was created in a different schema. Check SELECT DATABASE();
ERROR 1318, incorrect number of arguments	The CALL gave a different number of values than the procedure has parameters.
ERROR 1054, unknown column	Usually a parameter name spelled differently in the brackets and in the WHERE clause.
The CALL returns no rows	Check the value you passed. For student 9 or quiz9 no rows is the correct answer.